

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: **Informatyka techniczna (ITE)**
Specjalność: **Systemy informatyki w medycynie (IMT)**

PRACA DYPLOMOWA
INŻYNIERSKA

System wspierający diagnostykę jaskry

Veranika Kalosha

Opiekun pracy
dr inż. Jacek Cichosz

Słowa kluczowe: wspomaganie diagnostyki jaskry, przetwarzanie obrazu, aplikacja webowa

WROCŁAW 2026

STRESZCZENIE

Jaskra jest przewlekłą chorobą oczu, prowadzącą do nieodwracalnych uszkodzeń nerwu wzrokowego, a w skrajnych przypadkach – do całkowitej utraty wzroku. Ze względu na bezobjawowy przebieg we wczesnych stadiach, kluczowe znaczenie ma jej wczesne wykrycie. Celem pracy było zaprojektowanie i implementacja aplikacji webowej wspomagającej diagnozowanie jaskry na podstawie analizy obrazów dna oka.

System oparty na frameworku Django i architekturze MVT (Model-View-Template) umożliwia rejestrację lekarzy oraz przypisywanie zdjęć do profilów pacjentów. Do każdego obrazu lekarz może przypisać ocenę kliniczną oraz dokonać manualnego lub automatycznego pomiaru wskaźnika CDR (Cup-to-Disc Ratio), który jest jednym z najważniejszych parametrów w diagnostyce jaskry. Aplikacja wspiera automatyczną segmentację oraz wyznaczanie wskaźnika CDR z wykorzystaniem metod przetwarzania obrazu (przekształcenia morfologiczne, analiza kanałów barwnych, dynamiczne progowanie). Aplikacja umożliwia lekarzowi przegląd historii ocen, notatek i wyników CDR dla każdego pacjenta oraz generowanie raportów PDF zawierających kluczowe dane diagnostyczne.

Dodatkowo w pracy przedstawiono projekt architektury systemu, strukturę aplikacji oraz kierunki jej dalszego rozwoju, w tym możliwość integracji z systemami szpitalnymi oraz zastosowanie metod sztucznej inteligencji do klasyfikacji obrazów.

ABSTRACT

Glaucoma is a chronic eye disease that leads to irreversible damage to the optic nerve and, in severe cases, complete vision loss. Due to its asymptomatic nature in the early stages, early detection is crucial. The aim of this thesis was to design and implement a web application to support the diagnosis of glaucoma based on fundus image analysis.

The system, built using the Django framework and the MVT (Model-View-Template) architecture, enables physician registration and the assignment of fundus images to patient profiles. For each image, the physician can provide a clinical assessment and perform a manual or automatic measurement of the Cup-to-Disc Ratio (CDR), one of the key diagnostic parameters for glaucoma. The application supports automatic segmentation and CDR calculation using image processing techniques, including morphological operations, color channel analysis, and dynamic thresholding. The system also provides access to the history of assessments, notes, and CDR values for each patient and allows for generating PDF reports containing diagnostic data.

The thesis also presents the system architecture, project structure, and potential directions for further development, such as integration with hospital systems and the use of artificial intelligence methods for image classification.

Spis treści

Wprowadzenie	3
Cel i zakres pracy	3
1. Analiza problemu	5
1.1. Nowoczesne metody diagnostyki jaskry	5
1.2. Uzasadnienie potrzeby rozwoju systemu	5
2. Wymagania funkcjonalne	7
3. Narzędzia implementacyjne	9
3.1. Język programowania	9
3.2. Framework webowy	9
3.3. Baza danych	9
3.4. Środowisko konteneryzacji	10
3.5. Przetwarzanie i analiza obrazów	10
3.6. Frontend	10
3.7. Inne narzędzia	11
4. Projekt aplikacji	12
4.1. Architektura aplikacji	12
4.2. Struktura projektu aplikacji	13
4.3. Struktura bazy danych	15
4.4. Automatyczna segmentacja i ocena CDR	16
4.5. Opis działania najważniejszych komponentów	18
Przetwarzanie danych z API	18
Logowanie i autoryzacja użytkownika	19
Filtry kontrastu i jasności	21
Segmentacja obrazu dna oka	21
Przekształcenia morfologiczne	22
5. Interfejs użytkownika	24
6. Dalszy rozwój aplikacji	36

6.1. Wykorzystanie sztucznej inteligencji do klasyfikacji obrazów	36
6.2. Integracja z systemami szpitalnymi	36
6.3. Przetwarzanie asynchroniczne	36
Podsumowanie	37
Bibliografia	38
Spis listingów	41

WPROWADZENIE

Jaskra to choroba oczu charakteryzująca się poważnym uszkodzeniem nerwu wzrokowego. Według szacunków z 2013 roku, częstość występowania jaskry w populacji osób w wieku 40–80 lat wynosiła 3,54%, co oznaczało około 64,3 miliona chorych na całym świecie. Prognozuje się, że do 2040 roku liczba ta wzrośnie do około 111,82 miliona [1]. Jaskra prowadzi do nieodwracalnych uszkodzeń nerwu wzrokowego, a w skrajnych przypadkach – do całkowitej utraty wzroku. Kluczowe znaczenie ma więc wczesne wykrycie choroby. Problem polega jednak na tym, że jaskra we wczesnym stadium zwykle przebiega bezobjawowo, co znacznie utrudnia jej szybką diagnozę i leczenie.

Ze względu na poważne skutki choroby i obserwowany wzrost zachorowań, coraz większe znaczenie zyskują rozwiązania wspierające lekarzy w szybkim i precyzyjnym diagnozowaniu jaskry. Jednym z takich rozwiązań może być aplikacja informatyczna oparta na analizie obrazów dna oka.

Praca została podzielona na kilka rozdziałów:

- W rozdziale pierwszym przedstawiono charakterystykę problemu medycznego oraz współczesne metody diagnostyki jaskry.
- Rozdział drugi zawiera analizę potrzeb funkcjonalnych systemu.
- W rozdziale trzecim omówiono wykorzystane technologie i narzędzia programistyczne.
- Rozdział czwarty poświęcono projektowi architektury systemu oraz implementacji funkcjonalności aplikacji - zarówno od strony frontendowej, jak i backendowej - wraz z wykorzystanymi algorytmami do segmentacji obrazów i wyznaczania wskaźnika CDR.
- Rozdział piąty prezentuje wygląd i funkcjonalności interfejsu użytkownika aplikacji.
- Rozdział szósty dotyczy możliwych kierunków dalszego rozwoju.
- Na zakończenie przedstawiono podsumowanie całego projektu oraz wnioski końcowe.

CEL I ZAKRES PRACY

Celem pracy jest projekt i implementacja aplikacji webowej wspomagającej diagnozowanie jaskry poprzez wizualizację zdjęć dna oka, wybranych procedur ich przetwarzania w celu poprawy jakości i uwydatnienia użytecznych cech, umożliwienie pomiarów i ilościowego opisu cech obrazowych przydatnych w diagnozowaniu jaskry oraz zaprojektowanie bazy danych do przechowywania rekordów pacjentów oraz opisu zdjęć w celu usprawnienia zarządzania leczeniem.

Aby osiągnąć postawiony cel, należy rozwiązać następujące zadania:

- analiza charakterystycznych cech choroby oraz przegląd istniejących metod jej wykrywania, identyfikacja kluczowych parametrów diagnostycznych na podstawie wizualnej analizy obrazów;
- specyfikacja wymagań i założeń projektowych;
- projekt struktury systemu i jego kluczowych elementów;
- implementacja zestawu opcji wspomagających ocenę wizualną i przetwarzania obrazów, pomiar cech diagnostycznych;
- projekt części serwera, interfejsu API, interfejsu użytkownika zintegrowanego z bazą danych;
- testy na obrazowych danych dostępnych w internecie.

1. ANALIZA PROBLEMU

1.1. NOWOCZESNE METODY DIAGNOSTYKI JASKRY

Wczesna diagnostyka jaskry stanowi duże wyzwanie, ponieważ choroba ta może przebiegać bezobjawowo aż do momentu wystąpienia nieodwracalnych uszkodzeń nerwu wzrokowego. We współczesnej okulistyce stosuje się szereg metod diagnostycznych, w tym [2]:

- **Tonometria** – pomiar ciśnienia wewnątrzgałkowego,
- **Perymetria komputerowa** – ocena pola widzenia w celu wykrycia uszkodzeń,
- **Gonioskopia pośrednia** – badanie kąta przesączania (rogówkowo-tęczówkowego),
- **Optyczna koherentna tomografia (OCT)** — precyzyjna metoda skanowania siatkówki i tarczy nerwu wzrokowego, umożliwiająca ocenę ich struktury,
- **Oftalmoskopia** – wizualna ocena tarczy nerwu wzrokowego .

1.2. UZASADNIENIE POTRZEBY ROZWOJU SYSTEMU

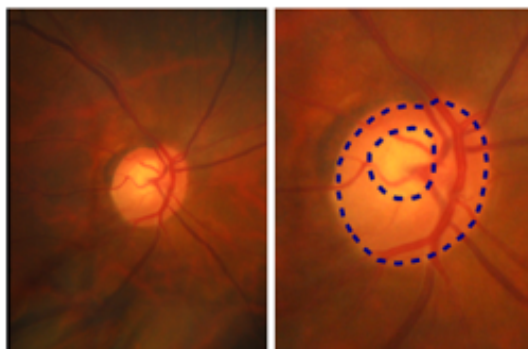
Pomimo skuteczności wcześniej wymienionych metod diagnostycznych, ich zastosowanie często wiąże się z koniecznością posiadania specjalistycznego sprzętu oraz wiedzy medycznej, co ogranicza ich dostępność w warunkach ograniczonych zasobów.

Wizualna ocena nerwu wzrokowego pozwala wykryć objawy jaskry, jednak czynniki takie jak podwyższone ciśnienie wewnątrzgałkowe, obciążenie genetyczne i inne objawy są jedynie wskaźnikami ryzyka — ich obecność nie przesądza o rozpoznaniu choroby [3]. Dodatkowo, podwyższone ciśnienie wewnątrzgałkowe często prowadzi do schorzeń innych niż jaskra (na przykład nadciśnienie oczne), a jaskrze nie zawsze towarzyszy zwiększone ciśnienie.

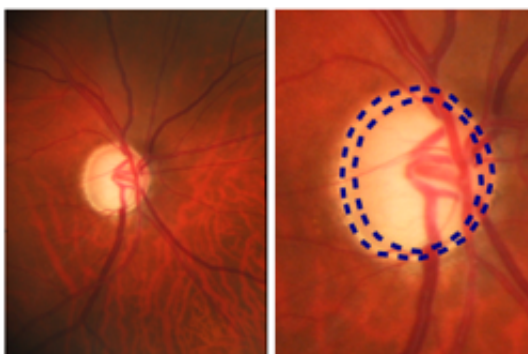
Perymetria pozwala jedynie ocenić konsekwencje choroby, a nie jej obecność — wskazuje, w jakim stopniu pole widzenia zostało zmniejszone przez jaskrę. Natomiast oftalmoskopia jest jedną z najskuteczniejszych metod diagnozowania jaskry, ponieważ ujawnia zniszczenie nerwu wzrokowego, niezależnie od tego, co było przyczyną.

Analiza obrazu dna oka wymaga od lekarza zlokalizowania obszarów tarczy i wgłębienia nerwu wzrokowego (jego wewnętrznego zachylenia, stosunkowo jasno wyrażonego na obrazie) oraz podkreślenia ich dokładnych granic. Obecność objawów jaskry można określić na podstawie stosunku wielkości średnicy wgłębienia do średnicy tarczy nerwu wzrokowego na powstałym obrazie (tzw. *cup-to-disk ratio*). Średnia wartość wskaźnika CDR u zdrowych

oczu wynosi $0,45 \pm 0,15$ (Rys.1), natomiast u pacjentów z jaskrą była istotnie wyższa i osiągała $0,80 \pm 0,16$ (Rys.2) [4].



Rysunek 1: Zdrowe oko ($CDR \approx 0.45$)



Rysunek 2: Oko z podejrzeniem jaskry ($CDR \approx 0.90$)

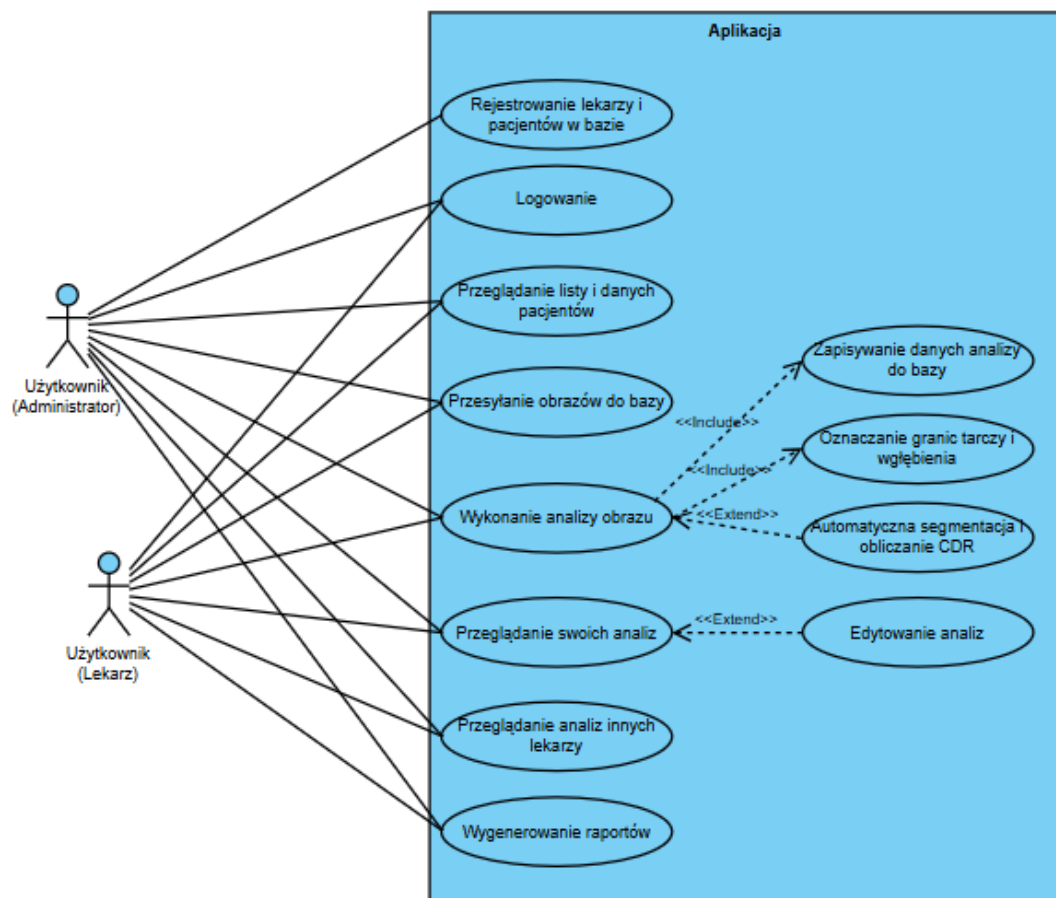
Z uwagi na złożoność i subiektywność tej analizy, istnieje potrzeba stworzenia zautomatyzowanego systemu, który wspomagałby proces diagnostyczny poprzez:

- identyfikację kluczowych struktur na obrazie,
- obliczanie wskaźników diagnostycznych,
- przejrzystą prezentację wyników dla użytkownika.

Opracowanie takiego systemu w formie aplikacji webowej z bazą danych i wsparciem dla konteneryzacji umożliwia stworzenie narzędzia dostępnego, skalowalnego i łatwego do wdrożenia w różnych środowiskach klinicznych.

2. WYMAGANIA FUNKCJONALNE

- Aplikacja umożliwia rejestrację i logowanie użytkowników z rolą lekarza lub administratora.
- Aplikacja umożliwia sortowanie listy wyników i wyszukiwanie pacjentów według ich nazwisk i imion.
- Aplikacja pozwala na dodawanie zdjęć dna oka do profilu pacjenta oraz ich przeglądanie i filtrowanie według oka (prawego lub lewego).
- Aplikacja umożliwia automatyczną segmentację obrazu i obliczenie wartości CDR (Cup-to-Disc Ratio) przy użyciu algorytmów przetwarzania obrazu.
- Aplikacja umożliwia zapisywanie wyników badania, w tym: wartość CDR, punkty zaznaczenia, notatki i opis kliniczny.
- Aplikacja umożliwia wygenerowanie i zapisanie raportu badania w formacie PDF na komputerze użytkownika.
- Lekarz może przeglądać listę pacjentów z podstawowymi danymi osobowymi (imię, nazwisko, data urodzenia, płeć).
- Podczas analizy zdjęcia lekarz może ręcznie zaznaczyć granicę tarczy i zagłębienia nerwu wzrokowego w celu obliczenia wartości CDR.
- Lekarz ma dostęp do historii wyników CDR dla wybranego pacjenta oraz może je przeglądać w formie interaktywnego wykresu.
- Lekarz może przeglądać wyniki badań wykonanych przez innych lekarzy.
- Lekarz może edytować tylko swoje badania.
- Administrator aplikacji ma możliwość działania jako dowolny lekarz.



Rysunek 3: Diagram przypadków użycia aplikacji.

Rysunek 3 przedstawia diagram przypadków użycia aplikacji. Diagram ten ilustruje główne funkcjonalności systemu oraz interakcje użytkowników z systemem.

3. NARZĘDZIA IMPLEMENTACYJNE

3.1. JĘZYK PROGRAMOWANIA

Python to interpretowany, wysokopoziomowy język programowania ogólnego zastosowania, który wyróżnia się przejrzystą i intuicyjną składnią przypominającą język angielski. Dzięki temu pozwala na szybkie tworzenie prototypów oraz pisanie czytelnego kodu. Jego bogata biblioteka standardowa oraz szeroki wybór zewnętrznych pakietów sprawiają, że jest doskonałym narzędziem do szybkiego rozwijania różnorodnych aplikacji [5]. Python znajduje zastosowanie m.in. w tworzeniu aplikacji webowych, analizie danych, uczeniu maszynowym, a także w przetwarzaniu obrazu i dźwięku. W opisanym projekcie do budowy aplikacji internetowej wykorzystano wersję 3.12.3 języka Python.

3.2. FRAMEWORK WEBOWY

Django to framework webowy oparty na języku Python, wykorzystywany do tworzenia nowoczesnych aplikacji internetowych. Umożliwia budowę bezpiecznych i skalowalnych serwisów WWW. Django zapewnia obsługę routingu, logiki aplikacji, integrację z bazami danych oraz generowanie dynamicznych stron HTML. Framework opiera się na wzorcu Model-View-Template (MVT), który pozwala na przejrzyste oddzielenie warstwy danych od warstwy prezentacji [6].

Django REST Framework to rozszerzenie frameworka Django, które pozwala w prosty sposób budować interfejsy API zgodne ze standardem REST. Umożliwia przesyłanie danych w formacie JSON między serwerem a aplikacjami frontendowymi lub mobilnymi. DRF oferuje gotowe komponenty do obsługi serializacji danych, paginacji, uwierzytelniania oraz kontroli dostępu. Dzięki temu tworzenie nowoczesnych, skalowalnych API staje się znacznie prostsze i bardziej efektywne [7].

3.3. BAZA DANYCH

PostgreSQL to zaawansowany system zarządzania relacyjnymi bazami danych. Umożliwia bezpieczne i wydajne przechowywanie dużych zbiorów danych, zapewniając jednocześnie zgodność ze standardem SQL. PostgreSQL wspiera również bardziej zaawansowane funkcje, takie jak operacje na danych przestrzennych czy obsługa replikacji. Jest często wykorzystywany w projektach, które wymagają niezawodności i elastyczności w przetwarzaniu danych [8].

3.4. ŚRODOWISKO KONTENERYZACJI

Docker to platforma służąca do tworzenia, wdrażania i uruchamiania aplikacji w tzw. kontenerach. Kontenery pozwalają na odizolowanie środowiska uruchomieniowego aplikacji, co zapewnia jej spójne działanie na różnych systemach. Docker ułatwia także automatyzację procesów wdrożeniowych oraz zarządzanie zależnościami projektu. Dzięki temu przyspiesza pracę zespołów deweloperskich i umożliwia łatwe przenoszenie aplikacji między środowiskami [9].

3.5. PRZETWARZANIE I ANALIZA OBRAZÓW

OpenCV to biblioteka do przetwarzania obrazów i analizy wideo. Oferuje szeroki zakres funkcji, takich jak detekcja obiektów, rozpoznawanie wzorców, segmentacja obrazu czy śledzenie ruchu. OpenCV jest wykorzystywana zarówno w projektach komercyjnych, jak i naukowych, zwłaszcza w dziedzinach takich jak sztuczna inteligencja czy robotyka [10].

NumPy to biblioteka języka Python przeznaczona do obliczeń numerycznych i pracy z dużymi zbiorami danych. Umożliwia efektywne operacje na tablicach i macierzach wielowymiarowych oraz oferuje bogaty zestaw funkcji matematycznych. NumPy jest powszechnie wykorzystywana w analizie danych, inżynierii oraz w projektach związanych z uczeniem maszynowym [11].

SciPy to zbiór bibliotek do zaawansowanych obliczeń naukowych i technicznych w Pythonie. Rozszerza możliwości biblioteki NumPy o funkcje do optymalizacji, integracji oraz przetwarzania sygnałów. Biblioteka ta jest często wykorzystywana w analizie danych i realizacji różnorodnych zadań inżynierskich [12].

Matplotlib to biblioteka języka Python służąca do tworzenia różnorodnych wykresów i wizualizacji danych. Umożliwia generowanie zarówno prostych wykresów liniowych, jak i skomplikowanych wizualizacji naukowych. Matplotlib wspiera integrację z innymi narzędziami analitycznymi i jest często wykorzystywana w analizie i prezentacji wyników badań [13].

3.6. FRONTEND

HTML to podstawowy język znaczników używany do tworzenia struktury stron internetowych. Pozwala definiować elementy takie jak nagłówki, akapity, obrazy, tabele czy odnośniki. HTML stanowi fundament każdej strony WWW i jest interpretowany przez przeglądarki internetowe [14].

CSS to język stylów wykorzystywany do definiowania wyglądu stron internetowych. Umożliwia precyzyjne określenie kolorów, czcionek, układu elementów oraz efektów

wizualnych. Dzięki CSS możliwe jest dostosowanie wyglądu strony do różnych urządzeń i rozdzielczości ekranu [15].

JavaScript to wszechstronny język programowania stosowany do tworzenia dynamicznych i interaktywnych stron internetowych. Pozwala na reagowanie na działania użytkownika, modyfikowanie zawartości strony bez przeładowywania oraz komunikację z serwerem w tle. JavaScript jest nieodłącznym elementem współczesnego web developementu [16].

Bootstrap to zestaw narzędzi do budowy nowoczesnych stron internetowych. Ułatwia tworzenie responsywnych interfejsów, które dobrze prezentują się na różnych urządzeniach. Zawiera bogaty zbiór gotowych komponentów, takich jak przyciski, formularze czy siatki, co znacząco przyspiesza proces projektowania. Jego główną zaletą jest łatwość użycia oraz pełne wsparcie dla standardów responsywnego projektowania [17].

3.7. INNE NARZĘDZIA

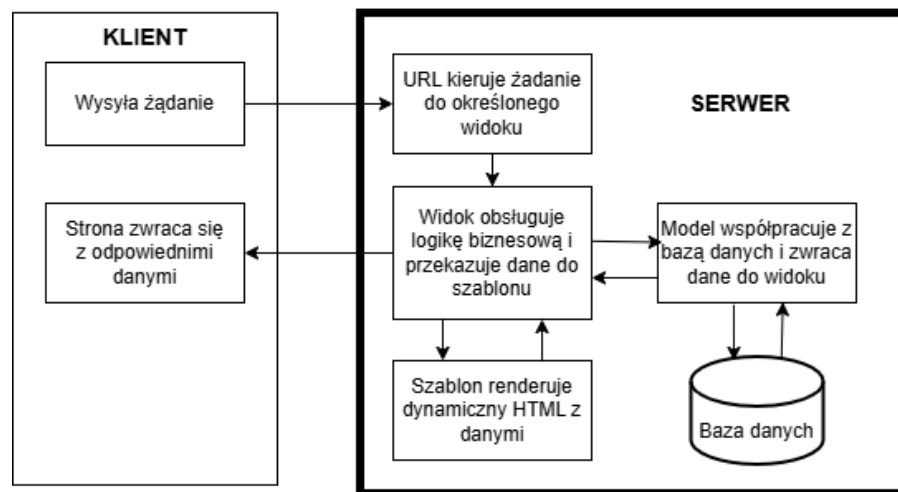
Git to system umożliwiający kontrolowanie wersji plików w projekcie. Pozwala on na zapisywanie kolejnych etapów pracy, cofanie zmian oraz współdzielenie kodu między programistami. Dzięki Git możliwa jest równoległa praca wielu osób nad jednym projektem, a cała historia zmian jest przechowywana w repozytorium, co ułatwia zarządzanie kodem [18].

Visual Studio Code to narzędzie programistyczne służące do pisania, debugowania i testowania kodu w wielu językach programowania. Umożliwia zarządzanie kontrolą wersji, tzw. SCM (Source Code Management), i obsługuje system Git. Jego główną zaletą jest wszechstronność oraz przejrzysty interfejs, który ułatwia użytkowanie. Program wykorzystywany jest do tworzenia stron internetowych, aplikacji webowych, usług sieciowych oraz aplikacji mobilnych [19].

4. PROJEKT APLIKACJI

4.1. ARCHITEKTURA APLIKACJI

Architektura aplikacji została oparta na wzorcu MVT (Model-View-Template), charakterystycznym dla frameworka Django. Architektura ta składa się z trzech głównych elementów: Modelu, Widoku i Szablonu, z których każdy pełni określoną rolę w działaniu aplikacji i współpracuje z pozostałymi. Ogólny schemat przepływu informacji pomiędzy warstwami przedstawia Rysunek 4.



Rysunek 4: Diagram przedstawiający przepływ danych w architekturze MVT Django [20].

Model – stanowi warstwę danych, odpowiada za przechowywanie, pobieranie i zarządzanie informacjami w bazie danych. Przykład struktury modelu danych pacjenta został przedstawiony na Listingu 1.

```
1 class Patient(models.Model):
2     first_name = models.CharField(max_length=100)
3     last_name = models.CharField(max_length=100)
4     date_of_birth = models.DateField()
5     gender = models.CharField(max_length=10)
```

Listing 1: Przykładowy model użyty w aplikacji.

Widok (View) – pełni rolę warstwy logiki, przetwarza dane, steruje przepływem informacji i decyduje, które dane mają trafić do interfejsu użytkownika. Widoki w Django mogą być

definiowane zarówno jako funkcje, jak i klasy. Przykład widoku służącego do pobierania danych pacjenta przedstawiono w Listingu 2.

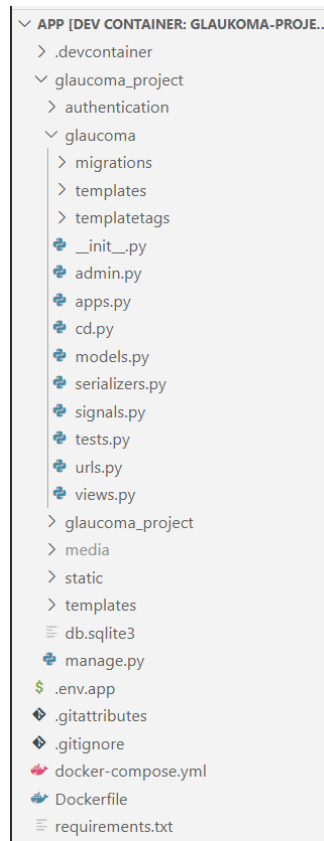
```
1 @login_required
2 def get_patient_details(request, patient_id):
3     patient = get_object_or_404(Patient, id=patient_id)
4     data = {
5         "first_name": patient.first_name,
6         "last_name": patient.last_name,
7         "gender": patient.gender,
8         "age": (timezone.now().date() - \
9             patient.date_of_birth).days // 365,
10    }
11    return JsonResponse(data)
```

Listing 2: Przykładowy widok użyty w aplikacji.

Szablon (Template) – to warstwa prezentacji, odpowiedzialna za generowanie i wyświetlanie treści HTML użytkownikowi [20].

4.2. STRUKTURA PROJEKTU APLIKACJI

Struktura projektu została zorganizowana zgodnie z konwencjami frameworka Django. Widok katalogów przedstawiono na Rysunku 5.



Rysunek 5: Struktura aplikacji.

.devcontainer – zawiera konfigurację kontenera deweloperskiego, który ułatwia uruchamianie projektu w jednolitym środowisku, niezależnie od systemu operacyjnego.

Glaucoma_project – główny katalog projektu Django.

Authentication – zawiera logikę autoryzacji i uwierzytelniania użytkowników.

Glaucoma – zawiera główną aplikację projektu odpowiedzialną za logikę z jaskrą. Zawiera:

Migrations – migracje bazy danych (automatycznie generowane pliki do aktualizacji schematu bazy);

Templates – szablony HTML specyficzne dla tej aplikacji;

cd.py – zawiera logikę automatycznej segmentacji oraz automatycznego obliczenia CDR;

models.py – definicje modeli danych;

serializers.py – konwertowanie danych między modelami a formatem JSON oraz walidacja przy komunikacji z API;

signals.py – zawiera sygnały Django, np. akcje po zapisaniu modelu;

urls.py – trasy URL przypisane do widoków;

views.py – funkcje odpowiedzialne za odpowiedzi na żądania HTTP.

Glaucoma_project – katalog głównych ustawień projektu Django.

Media – katalog przechowujący zdjęcia dna oka pacjentów.

Static – zawiera wszystkie pliki CSS, JavaScript.

manage.py – główne narzędzie CLI Django – służy do uruchamiania serwera, migracji itp.

env.app – konfiguracja bazy danych.

.gitattributes, .gitignore – pliki konfiguracyjne dla Git – zarządzają wersjonowaniem i wykluczeniami plików.

docker-compose.yml, Dockerfile – pliki służące do uruchamiania projektu w kontenerze Docker, co zapewnia przenośne i powtarzalne środowisko pracy.

requirements.txt – lista bibliotek Python potrzebnych do uruchomienia projektu.

4.3. STRUKTURA BAZY DANYCH

Do zaprojektowania bazy danych aplikacji wykorzystano mechanizm modeli oferowany przez framework Django. Modele te definiowane są w języku Python i umożliwiają tworzenie struktur odpowiadających tabelom w relacyjnej bazie danych. Dzięki wbudowanemu w Django mechanizmowi ORM (Object-Relational-Mapping) możliwe jest zarządzanie danymi w sposób obiektowy, bez konieczności bezpośredniego pisania zapytań SQL. Struktura modeli odzwierciedla logiczne powiązania między obiektami – np. Relacje jeden-do-wielu lub wiele-do-jednego – co znacząco upraszcza proces implementacji i późniejszej modyfikacji schematu bazy danych.

W projekcie wykorzystano relacyjną bazę danych PostgreSQL, która została wybrana ze względu na swoją niezawodność, skalowalność oraz pełną kompatybilność z Django ORM. W skład struktury danych wchodzi następujące modele:

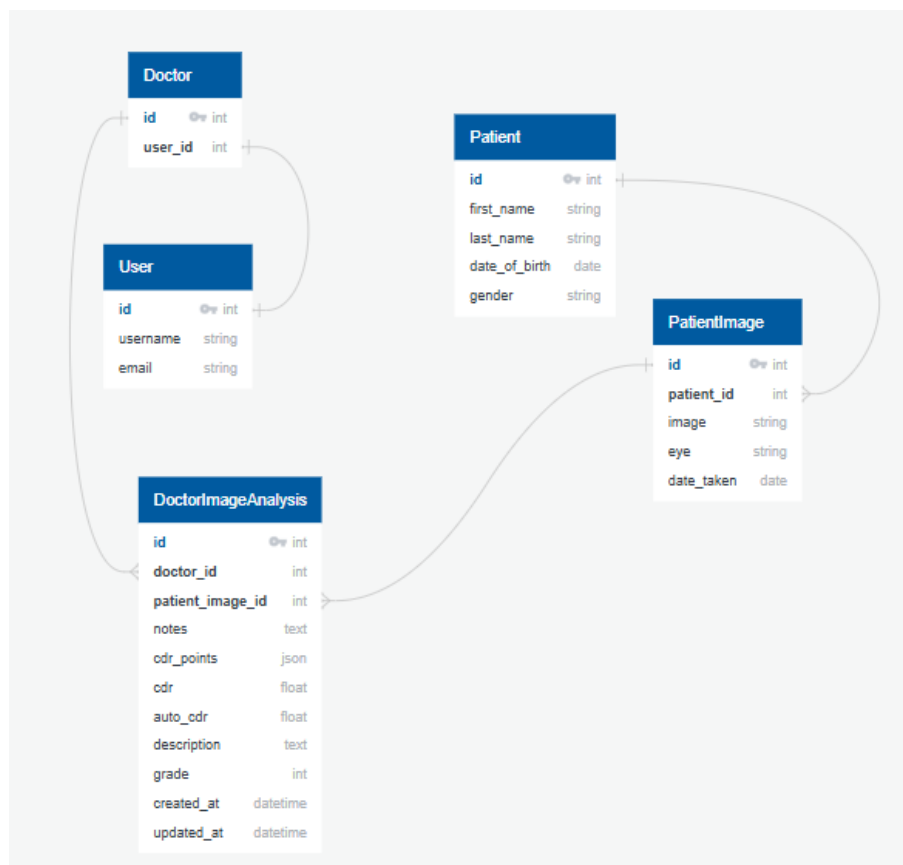
Doctor reprezentuje lekarza i jest powiązany relacją jeden-do-jednego z kontem użytkownika (User),

Patient przechowuje podstawowe informacje o pacjentach, takie jak imię, nazwisko, data urodzenia oraz płeć,

PatientImage zawiera linki na obrazy dna oka przypisane do pacjenta, wraz z datą ich wykonania oraz informacją, którego oka dotyczą,

DoctorImageAnalysis rejestruje wyniki analizy obrazu przeprowadzonej przez konkretnego lekarza, w tym ocenę, opis oraz wskaźnik CDR.

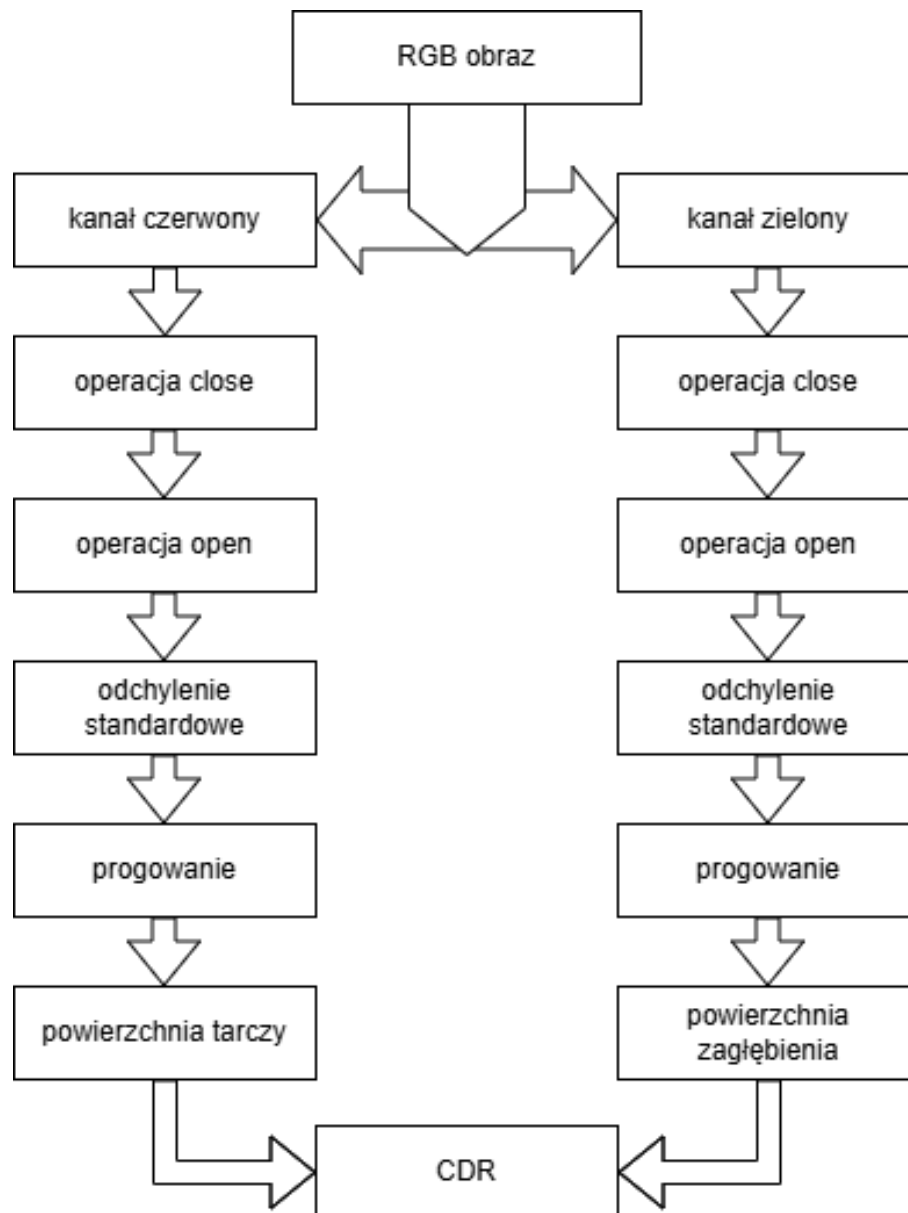
Diagram encji-relacji (ERD) został przygotowany przy użyciu narzędzia [quickdiagram.com](https://www.quickdiagrams.com/). (Rys. 6).



Rysunek 6: Diagram ERD bazy danych.

4.4. AUTOMATYCZNA SEGMENTACJA I OCENA CDR

W aplikacji zaimplementowano moduł umożliwiający automatyczną ocenę CDR, która stanowi kluczowy parametr w diagnostyce jaskry. Proces analizy obrazu przedstawia poniższy diagram blokowy (Rys.7):



Rysunek 7: Schemat blokowy przetwarzania obrazu do obliczania wskaźnika CDR [21].

Analiza kanałów RGB wykazała, że tarcza nerwu wzrokowego najlepiej widoczna jest w czerwonym kanale, natomiast granica między tarczą a zagłębieniem lepiej uwidacznia się w zielonym. Dlatego do segmentacji tarczy wykorzystano obraz z kanału czerwonego, a do segmentacji zagłębienia – z kanału zielonego.

Aby zwiększyć dokładność detekcji, usunięto naczynia krwionośne przy użyciu operacji morfologicznych: *close* i *open*. Operacje te wygładzają krawędzie i eliminują drobne artefakty (Rys. 7).

Po przetworzeniu obrazów obliczono odchylenie standardowe i na jego podstawie dobrano progi binarnej segmentacji.

Na podstawie zbinaryzowanych obrazów obliczono pola tarczy i zagłębienia przez zliczenie jasnych pikseli. [21]

4.5. OPIS DZIAŁANIA NAJWAŻNIEJSZYCH KOMPONENTÓW

Przetwarzanie danych z API

Aplikacja intensywnie wykorzystuje komunikację między frontendem a backendem poprzez API. Wymiana danych realizowana jest asynchronicznie za pomocą technologii JavaScript (fetchAPI), co pozwala użytkownikowi na płynne działanie interfejsu bez konieczności przeładowywania strony. Po pobraniu oceny (np. "Glaucoma"), aplikacja frontendowa przesyła dane do serwera przy pomocy zapytania typu POST (List. 3).

```
1 const response = await fetch(  
2     "/glaucoma/patients/" + currentPatientId +  
3     "/images/" + imageId +  
4     "/save_grade/",  
5     {  
6         method: "POST",  
7         headers: {  
8             "Content-Type": "application/json",  
9         },  
10        body: JSON.stringify({ grade })  
11    }  
12 );
```

Listing 3: Zapisywanie oceny zdjęcia do API (home.js).

Backend Django odbiera dane przesłane przez JavaScript w formacie JSON, odczytuje ocenę i zapisuje ją do bazy danych (List. 4).

```
1 @csrf_exempt  
2 @login_required  
3 def save_grade_view(request, patient_id, image_id):  
4     if request.method == "POST":  
5         try:  
6             image = get_object_or_404(PatientImage, \  
7                 id=image_id)  
8  
9             doctor = request.user.doctor  
10  
11            data = json.loads(request.body)  
12            grade_value = data.get("grade")  
13  
14            if grade_value == "null":  
15                grade_value = None  
16
```

```

17         analysis, created = \
18         DoctorImageAnalysis.objects.get_or_create(
19             patient_image=image,
20             doctor=doctor,
21             defaults={"grade": grade_value},
22         )
23         if not created:
24             analysis.grade = grade_value
25             analysis.save()
26
27         return JsonResponse
28         \ ({"message": "Grade saved successfully!"})
29
30     except Exception as e:
31         return JsonResponse({"error": str(e)}, \
32             status=500)
33
34     return JsonResponse({"error": "Invalid method."}, \
35         status=400)

```

Listing 4: Widok Django zapisujący ocenę lekarza.

Logowanie i autoryzacja użytkownika

W aplikacji zastosowano standardowy mechanizm logowania Django opartego na modelu User. Logowanie realizowane jest w widoku login_view. W przypadku żądania typu POST pobierane są dane z formularza logowania, które następnie są przekazywane do funkcji authenticate(). Jeżeli dane są prawidłowe, funkcja login() zapisuje dane sesji i użytkownik zostaje przekierowany do strony głównej systemu. W przeciwnym przypadku otrzymuje komunikat o nieprawidłowych danych (List.5).

```

1 def login_view(request):
2     if request.method == 'POST':
3         username = request.POST.get('username')
4         password = request.POST.get('password')
5         user = authenticate \
6         (request, username=username, password=password)
7
8         if user:
9             login(request, user)
10            return redirect('/glaucoma/home')

```



```

11         else:
12             return JsonResponse \
13                 ({'message': 'Invalid username or password'}, \
14                  status=401)
15
16     return render(request, 'authentication/login.html')

```

Listing 5: Widok logowania (views.py).

Formularz logowania jest prosty i zawiera pola do wprowadzenia nazwy użytkownika i hasła (List. 6). Po przesłaniu danych odwołuje się do powyższego widoku.

```

1 <form method="post" id="login-form">
2     {% csrf_token %}
3     <label for="username">Username:</label>
4     <input type="text" id="username" name="username" required>
5
6     <label for="password">Password:</label>
7     <input type="password" id="password" \
8         name="password" required>
9     <button type="submit">Login</button>
10 </form>

```

Listing 6: Formularz logowania (login.html).

Dostęp do większości widoków w systemie zabezpieczony jest dekoratorem `@login_required`, który wymusza logowanie przed wejściem na daną stronę. Próba dostępu bez zalogowania przekierowuje użytkownika do formularza logowania (List. 7).

```

1 @login_required
2 def home_page(request):
3     return render(request, "glaucoma/home.html")

```

Listing 7: Przykład zabezpieczonego widoku (views.py).

Poza logowaniem, w aplikacji zaimplementowano również prostą autoryzację. System sprawdza, czy aktualnie zalogowany użytkownik (lekarz) jest właścicielem analizy – czyli czy może ją edytować. Jeżeli nie, interfejs użytkownika automatycznie przełącza się w tryb tylko do odczytu (List. 8).

```

1 is_readonly = selected_doctor != current_doctor

```

Listing 8: Warunek autoryzacji edycji danych.

Taka implementacja pozwala lekarzom przeglądać analizy wykonane przez innych, ale edytować mogą wyłącznie własne wpisy. Gwarantuje to zarówno przejrzystość pracy zespołowej, jak i bezpieczeństwo danych pacjentów.

Filtry kontrastu i jasności

W aplikacji zaimplementowano możliwość dynamicznego dostosowywania jasności i kontrastu obrazu w przeglądarce, co pozwala lekarzowi lepiej dostrzegać szczegóły struktury dna oka. Operacja ta nie modyfikuje oryginalnego obrazu – zmiany są stosowane wyłącznie wizualnie.

```
1 function applyFilters() {  
2     const b = parseInt(brightnessSlider.value);  
3     const c = parseInt(contrastSlider.value);  
4     ctx.filter = `brightness(${100 + b}%) \\  
5     contrast(${100 + c}%)`;\  
6     redrawImage(true);\  
7 }
```

Listing 9: Funkcja modyfikacji jasności i kontrastu obrazu.

Wartości suwaków jasności i kontrastu są przeliczane na procenty i stosowane do kontekstu renderowania canvas za pomocą właściwości `ctx.filter` (List. 9). Następnie obraz jest rysowany ponownie z uwzględnieniem wprowadzonych zmian.

Segmentacja obrazu dna oka

Segmentacja ma na celu wydzielenie dwóch kluczowych struktur w obrazie dna oka: tarczy nerwu wzrokowego oraz wgłębienia. W przedstawionym kodzie segmentacja opiera się na analizie poszczególnych kanałów koloru (czerwonego - Aro i zielonego - Ago) oraz progu intensywności jasności pikseli. Do przetwarzania wykorzystana jest technika CLAHE (ang. Contrast Limited Adaptive Histogram Equalization), która lokalnie zwiększa kontrast obrazu, ułatwiając dalszą segmentację (List. 10).

```
1 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(10, 10))
```

Listing 10: Zastosowanie CLAHE w przetwarzaniu obrazu.

Następnie za pomocą filtrów Gaussa obliczane jest odchylenie standardowe filtra (STDf), które w połączeniu z odchyleniem standardowym (SDr) służy do dynamicznego wyznaczenia progu segmentacji Thr. W wyniku otrzymujemy binarną maskę tarczy (Dd). Analogicznie dla kanału zielonego jest wyznaczana maska Dc odpowiadająca wgłębieniu (List. 11).

```
1 def segment(image, plot_hist=False, plot_seg=False):  
2     image = crop_to_roi(image)  
3     Abo, Ago, Aro = cv2.split(image)  
4     Aro = clahe.apply(Aro)  
5
```

```

6     M = 60
7     filt = signal.windows.gaussian(M, std=6)
8     STDf = filt.std()
9
10    Ar = Aro - Aro.mean() - Aro.std()
11    SDr = Ar.std()
12    Thr = 0.5 * M - STDf - SDr
13
14    M = 20
15    filter_cup = signal.windows.gaussian(M, std=6)
16    STDf = filter_cup.std()
17
18    Ag = Ago - Ago.mean() - Ago.std()
19    Mg = Ag.mean()
20    SDg = Ag.std()
21    Thg = 0.5 * M + 2 * STDf + 2 * SDg + Mg
22
23    r, c = Ag.shape
24    Dd = np.zeros((r, c), dtype=np.uint8)
25    Dc = np.zeros((r, c), dtype=np.uint8)
26
27    # Disc
28    for i in range(1, r):
29        for j in range(1, c):
30            Dd[i, j] = 255 if Ar[i, j] > Thr else 0
31
32    # Cup
33    for i in range(1, r):
34        for j in range(1, c):
35            Dc[i, j] = 255 if Ag[i, j] > Thg else 0
36
37    return Dd, Dc

```

Listing 11: Funkcja segmentacji obrazu.

Przekształcenia morfologiczne

Aby uzyskać gładzsze i bardziej regularne kontury obiektów, zastosowano serię przekształceń morfologicznych (List. 12). Są to operacje, które modyfikują kształt obiektów w obrazie binarnym:

- **Zamykanie (morphological closing)** – wypełnia małe dziury i łączy sąsiednie obiekty,

- **Otwarcie (morphological opening)** – usuwa małe obiekty (szumy) i wygładza kontur.

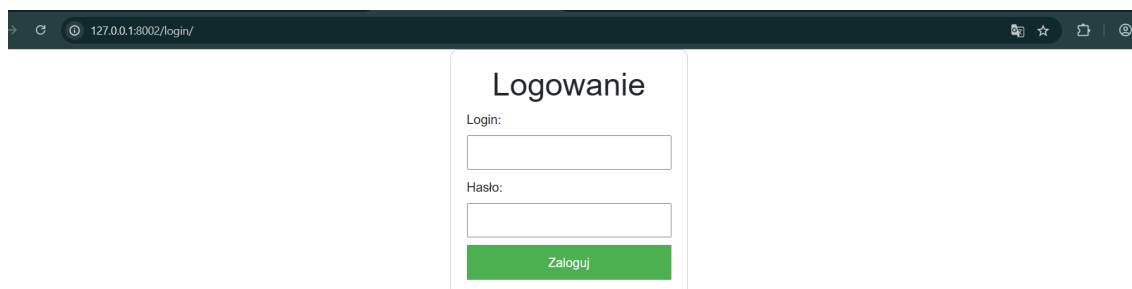
```
1 R1 =cv2.morphologyEx(cup,cv2.MORPH_CLOSE,\n2 cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(2,2)))\n3 r1 = cv2.morphologyEx(R1,cv2.MORPH_OPEN,\n4 cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(7,7)))
```

Listing 12: Przekształcenia morfologiczne.

Dzięki zastosowaniu tych operacji możliwe jest uzyskanie dobrze odwzorowanych konturów tarczy i wgłębienia, które można dalej analizować za pomocą elips dopasowanych metodą `fitEllipse`.

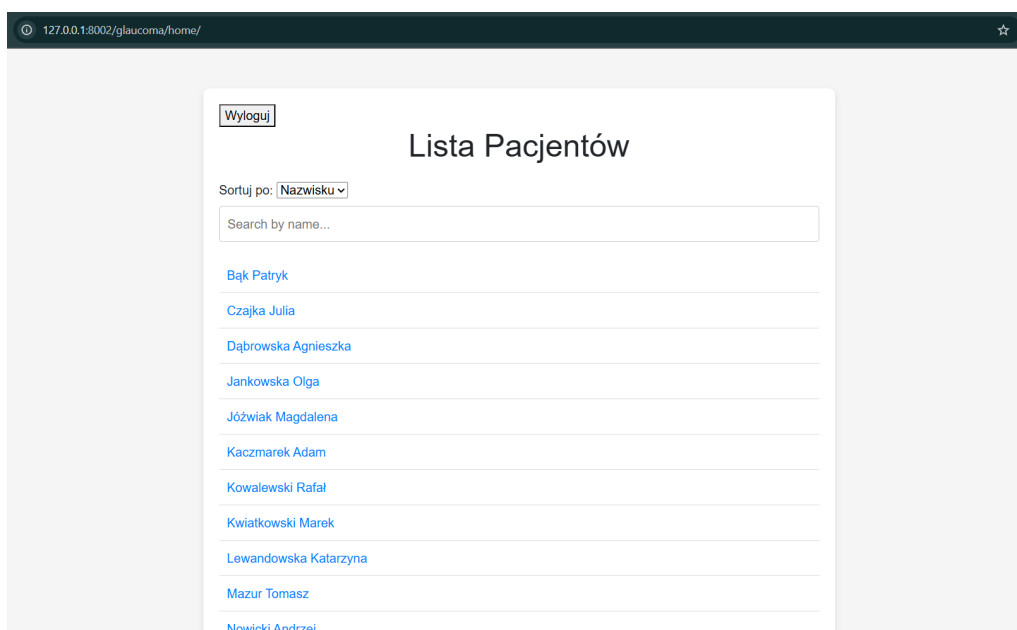
5. INTERFEJS UŻYTKOWNIKA

Po wprowadzeniu adresu URL w przeglądarce użytkownik zostaje przekierowywany do strony logowania (Rys.8), która stanowi punkt startowy aplikacji i umożliwia dostęp do systemu po poprawnym uwierzytelnieniu .



Rysunek 8: Strona logowania użytkownika.

Po poprawnym zalogowaniu się użytkownik (lekarz) zostaje przekierowany do głównego widoku aplikacji – panelu z listą pacjentów (Rys.9).



Rysunek 9: Główna strona aplikacji.

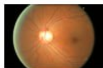
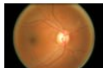
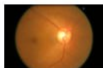
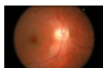
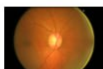
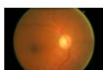
Interfejs użytkownika umożliwia szybkie wyszukiwanie pacjentów, sortowanie listy według imienia lub nazwiska oraz przeglądanie szczegółowych informacji o wybranym pacjencie. Dane pacjenta prezentowane są w formie przejrzystego panelu modalnego zawierającego m. in. dane osobowe, płeć oraz wiek (Rys.10).

The screenshot shows a web application interface. At the top, there is a 'Wyloguj' button. Below it is a header 'Lista Pacjentów'. A modal window titled 'Dane pacjenta' is open, displaying patient information: 'Imię: Patryk', 'Nazwisko: Bąk', 'Płeć: mężczyzna', and 'Wiek: 37'. Below this is a 'Statystyka' bar. Under the 'Obrazy' section, there is a table with columns: 'Data', 'Oko' (with a dropdown menu showing 'Oczy'), 'Obraz', 'Ostatnia aktualizacja', 'CDR (Moje CDR/ śr. CDR)', and 'Ocena' (with a dropdown menu showing 'Moja ocena'). Below the table is a 'Prześlij obraz' section with a date input (dd.mm.gggg), an 'Oko' dropdown (showing 'Lewe'), an 'Obraz' file selection button (labeled 'Выберите файл'), and a 'Prześlij' button.

Rysunek 10: Panel z danymi pacjenta.

W tym samym widoku użytkownik może przeglądać dostępne obrazy dna oka konkretnego pacjenta, przysyłać nowe zdjęcia oraz filtrować wyniki według oka (Rys.11, 12).

Obrazy

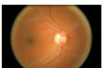
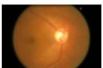
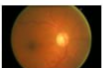
Data	Oko Oczy ▾	Obraz	Ostatnia aktualizacja	CDR (Moje CDR/ Śr. CDR)	Ocena Moja ocena ▾	
2025-05-08	Prawe		09.06.2025, 17:06:53	0.6 / 0.60	wybierz.. ▾	Wygeneruj raport
2025-04-24	Lewe		13.06.2025, 02:31:42	0.63 / 0.64	Jaskra ▾	Wygeneruj raport
2025-01-14	Lewe		13.06.2025, 02:31:36	0.55 / 0.52	Zdrowe ▾	Wygeneruj raport
2025-01-14	Prawe		13.06.2025, 02:31:29	0.51 / 0.55	Zdrowe ▾	Wygeneruj raport
2024-05-13	Prawe		13.06.2025, 02:31:22	0.6 / 0.63	Brak diagnozy ▾	Wygeneruj raport
2024-01-09	Lewe		13.06.2025, 02:31:09	0.45 / 0.46	Zdrowe ▾	Wygeneruj raport

Prześlij obraz

Data: ☐
Oko:
Obraz:

Rysunek 11: Panel z danymi obrazów przed sortowaniem.

Obrazy

Data	Oko Lewe oko ▾	Obraz	Ostatnia aktualizacja	CDR (Moje CDR/ Śr. CDR)	Ocena Moja ocena ▾	
2025-04-24	Lewe		13.06.2025, 02:31:42	0.63 / 0.64	Jaskra ▾	Wygeneruj raport
2025-01-14	Lewe		13.06.2025, 02:31:36	0.55 / 0.52	Zdrowe ▾	Wygeneruj raport
2024-01-09	Lewe		13.06.2025, 02:31:09	0.45 / 0.46	Zdrowe ▾	Wygeneruj raport

Prześlij obraz

Data: ☐
Oko:
Obraz:

Rysunek 12: Panel z danymi obrazów po sortowaniu.

Zadaniem tabeli z obrazami dna oka pacjenta jest umożliwienie lekarzowi szybkiego przeglądu wszystkich dostępnych badań obrazowych konkretnego pacjenta oraz dostarczenie informacji diagnostycznych w przejrzystej formie.

Tabela zawiera następujące dane:

- **Data** – data wykonania fundus-zdjęcia,
- **Oko** – określenie oka, którego dotyczy obraz,
- **Obraz** – miniatura zdjęcia, która po kliknięciu prowadzi do widoku analizy,
- **Ostatnia aktualizacja** – data ostatniej modyfikacji lub zapisu analizy tego obrazu,

- **CDR** – wskaźnik Cup_to_Disc Ratio, który może zawierać zarówno wartość obliczoną ręcznie przez lekarza, jak i średnią z analiz innych lekarzy,
- **Ocena** – diagnoza przypisana do obrazu.

Data	Oko Oczy	Obraz	Ostatnia aktualizacja	CDR (Moje CDR/ Śr. CDR)	Ocena Moja ocena	
2025-05-08	Prawe		11.06.2025, 16:39:43	0.6 / 0.60	Zdrowe	Wygeneruj raport
2025-04-24	Lewe		09.06.2025, 19:14:15	0.65 / 0.64	wyberz.. wyberz.. Zdrowe	Wygeneruj raport
2025-01-14	Lewe		09.06.2025, 16:14:28	0.48 / 0.52	Brak diagnozy Jaskra	Wygeneruj raport

Rysunek 13: Wystawienie oceny przez lekarza.

Lekarz posiada możliwość przypisania oceny do każdego obrazu dna oka. Proces ten odbywa się za pomocą rozwijanego menu w kolumnie „Ocena” (Rys. 13). Lekarz ma do wyboru trzy możliwe diagnozy:

- **Zdrowe** – oznacza brak patologii w analizowanym obrazie, zgodny z prawidłową anatomią,
- **Brak diagnozy** – wybierana wtedy, gdy lekarz nie podejmuje decyzji diagnostycznej w danym momencie,
- **Jaskra** – wskazuje, że na podstawie analizy obrazu istnieją cechy charakterystyczne dla jaskry, takie jak podwyższony wskaźnik CDR lub zmiany strukturalne w obrębie tarczy nerwu wzrokowego.

Po wybraniu oceny, wartość ta zostaje zapisana do bazy danych.

Obrazy

Data	Oko Oczy	Obraz	Ostatnia aktualizacja	CDR (Moje CDR/ Śr. CDR)	Ocena Śr. ocena	
2025-05-08	Prawe		No data	— / 0.60		Wygeneruj raport
2025-04-24	Lewe		No data	— / 0.64		Wygeneruj raport
2025-01-14	Lewe		No data	— / 0.52		Wygeneruj raport
2025-01-14	Prawe		No data	— / 0.55		Wygeneruj raport
2024-05-13	Prawe		No data	— / 0.63		Wygeneruj raport
2024-01-09	Lewe		No data	— / 0.46		Wygeneruj raport

Prześlij obraz

Rysunek 14: Widok tabeli obrazów w trybie wyświetlania ocen średnich.

Na Rysunku 14 przedstawiono widok tabeli obrazów w trybie wyświetlania ocen średnich, uzyskanych na podstawie diagnoz wystawionych przez wielu lekarzy. W kolumnie „Ocena” użytkownik może wybrać z listy rozwijanej opcję „Śr. ocena” (średnia ocena). Po jej aktywowaniu interfejs dynamicznie dopasowuje kolor tła każdego wiersza, co stanowi wizualne wsparcie w interpretacji wyników.

Każdy rząd odpowiada jednemu obrazowi i w zależności od rodzaju średniej oceny jest podświetlany odpowiednim kolorem:

- Zielony oznacza, że obraz został oceniony jako „Zdrowe” (brak zmian chorobowych).
- Czerwony sygnalizuje rozpoznanie „Jaskra” – potencjalnie niebezpieczny przypadek wymagający dalszej diagnostyki lub interwencji.
- Szary wskazuje, że nie została wystawiona jednoznaczna diagnoza (brak jednoznacznej decyzji).

Dzięki tej funkcjonalności lekarz może szybko zorientować się, które obrazy wymagają szczególnej uwagi – np. obrazy zaznaczone na czerwono mogą zostać natychmiast poddane dokładniejszej analizie, natomiast zielone można uznać za niebudzące niepokoju.

Lekarz: Jan Kowalski	2025-06-13
Pacjent: Patryk Bąk Płeć: męska Data urodzenia: 02-04-1988 Wiek: 37	
	
Oko: Right Data: 08-05-2025	
Opis: CDR ≈ 0.60 Obraz stabilny, nie zaobserwowano istotnych zmian względem poprzedniego badania.	
Ocena: Healthy	
Wynik: Brak cech jaskry – tarcza fizjologiczna. Zalecana kontrola za 6 miesięcy. Obraz zgodny z normą dla danego wieku pacjenta.	

Rysunek 15: Wygenerowany raport

Po kliknięciu przycisku "Wygeneruj raport" system automatycznie tworzy dokument PDF (Rys. 15), który zostaje pobrany na komputer użytkownika. Raport zawiera najważniejsze informacje dotyczące konkretnego badania:

- Dane lekarza i pacjenta,

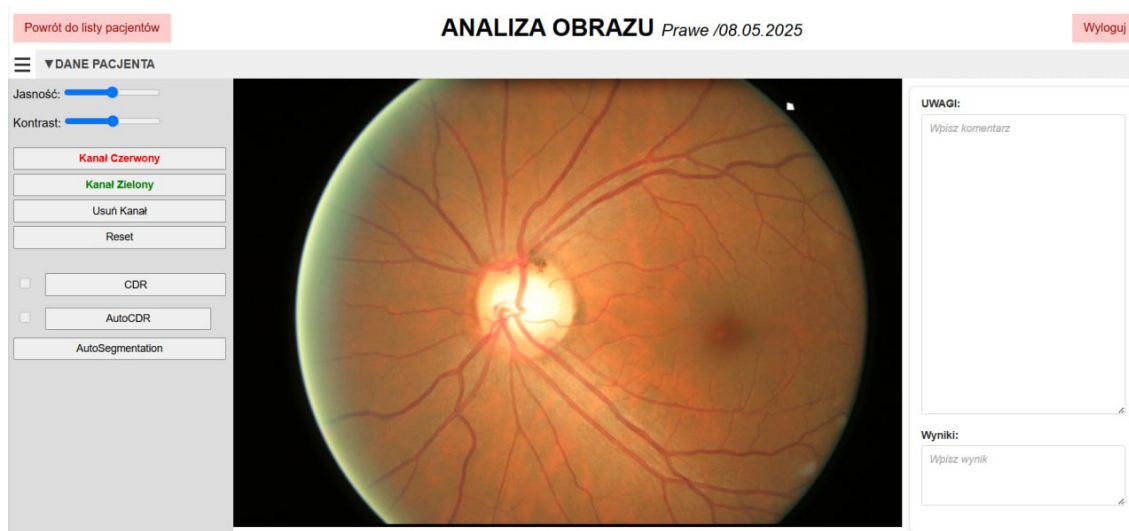
- Obraz oka, którego dotyczy analiza,
- Opis, ocenę i wynik przeprowadzonego badania.



Rysunek 16: Panel statystyk wartości CDR.

Panel modalny prezentujący statystyki wartości wskaźnika CDR oraz AutoCDR w podziale na lewe i prawe oko pacjenta (Rys. 16). Statystyki służą lekarzowi do wizualnego śledzenia postępów lub zmian w strukturze nerwu wzrokowego na przestrzeni czasu.

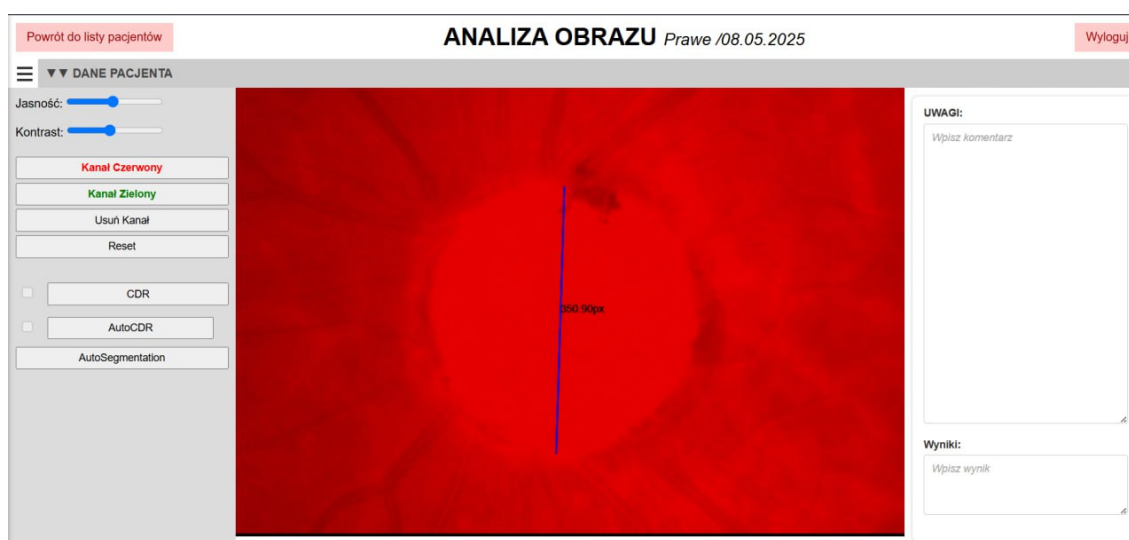
Po kliknięciu na obraz przechodzimy do strony służącej lekarzowi do szczegółowego badania obrazu dna oka pacjenta. Widok `image_analysis.html` został zaprojektowany w sposób interaktywny, umożliwiający jednocześnie przeglądanie obrazu, nanoszenie pomiarów oraz zapisywanie wyników diagnostycznych.



Rysunek 17: Strona badania obrazu.

Centralna część ekranu prezentuje obraz oka (prawego) wykonany dnia 08.05.2025 (Rys. 17). Obraz pozwala na interaktywne:

- Powiększanie i przesuwanie,
- Zaznaczenie punktów w celu obliczenia wskaźnika CDR,
- Oglądanie obrazu w wybranym kanale (np. czerwonym) (Rys. 18).

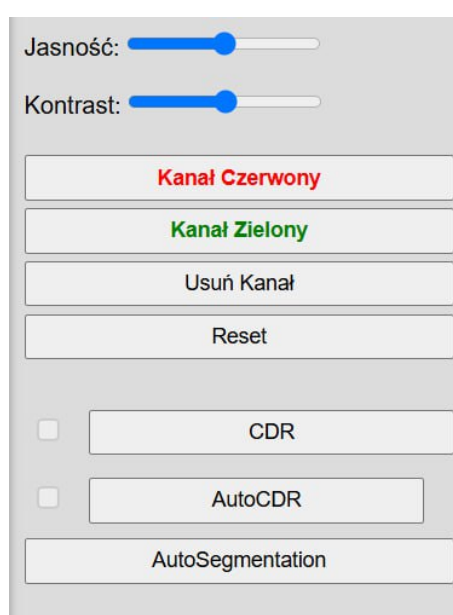


Rysunek 18: Powiększony obraz dna oka w kanale czerwonym z naniesionymi punktami granic tarczy nerwu wzrokowego (Optic Disc).



Rysunek 19: Sekcja „Dane pacjenta”.

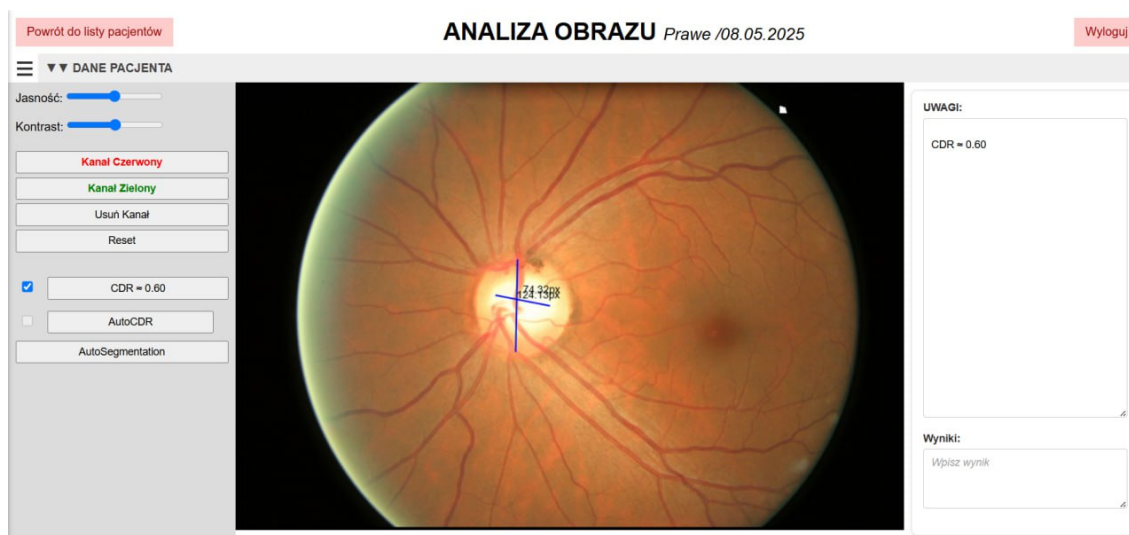
Sekcja „DANE PACJENTA” – rozwijana, zawiera podstawowe informacje o pacjencie (Rys. 19).



Rysunek 20: Panel narzędzi po lewej stronie.

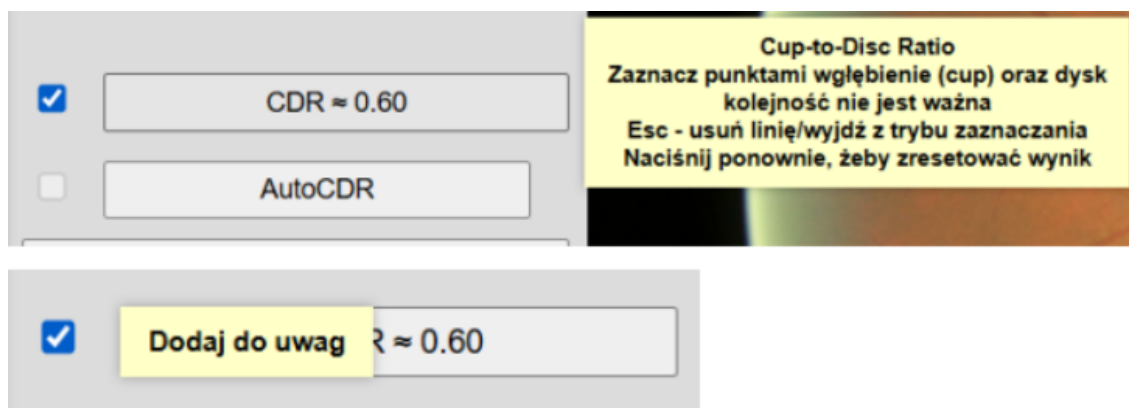
Panel narzędzi po lewej stronie zawiera (Rys. 20):

- Suwaki do regulacji jasności i kontrastu,
- Przełączniki kanałów kolorów umożliwiają wybór odpowiedniego widoku obrazu w celu ułatwienia analizy struktur anatomicznych. Zgodnie z opisem w punkcie 4.4, do wyznaczenia granic tarczy nerwu wzrokowego zastosowano kanał czerwony, natomiast do precyzyjnego oznaczenia granic wgłębienia – kanał zielony,
- Narzędzia analizy: **CDR**, **AutoCDR**, **AutoSegmentacja**,
- Przyciski resetowania i usuwania kanałów.



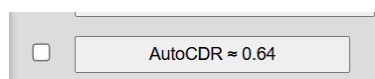
Rysunek 21: Wyznaczanie wskaźnika CDR.

Po kliknięciu przycisku „CDR” użytkownik uzyskuje możliwość zaznaczenia granic tarczy nerwu wzrokowego oraz wgłębienia. W celu wykonania pomiaru należy wskazać po dwa punkty dla każdej z linii – odpowiednio dla dysku i wgłębienia. Po zaznaczeniu obu par punktów linie zostają automatycznie narysowane, a wartość wskaźnika CDR zostaje obliczona i wyświetlona bezpośrednio na przycisku. Równocześnie aktywuje się pole wyboru (checkbox), które umożliwia zapisanie uzyskanej wartości do pola „Uwagi” (Rys. 21).



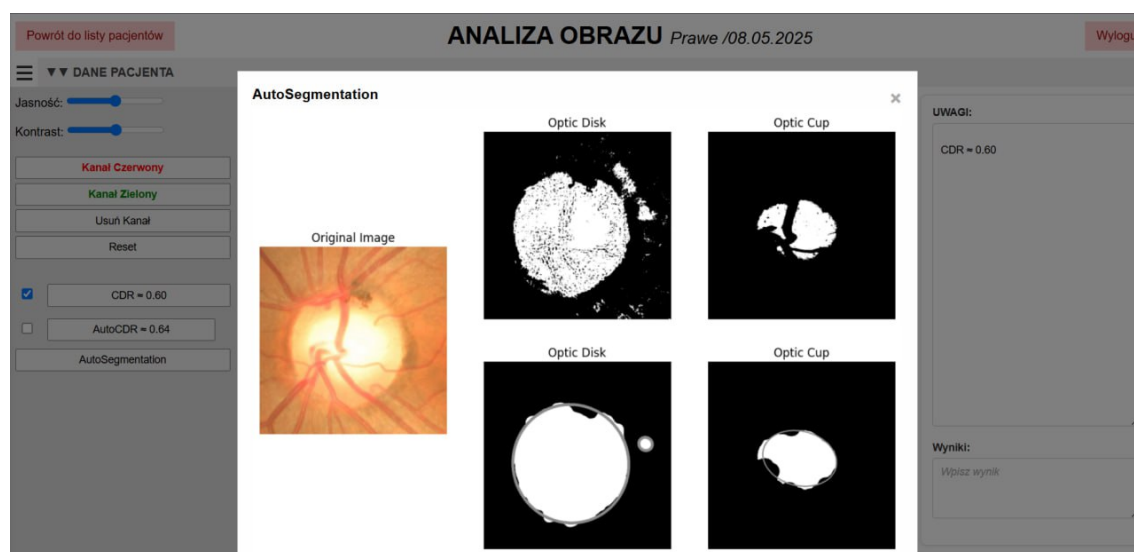
Rysunek 22: Podpowiedzi korzystania z narzędzi pomiarowych.

Po najechnaniu kursorem na poszczególne przyciski, pojawiają się podpowiedzi ułatwiające prawidłowe korzystanie z narzędzi pomiarowych (Rys. 22).



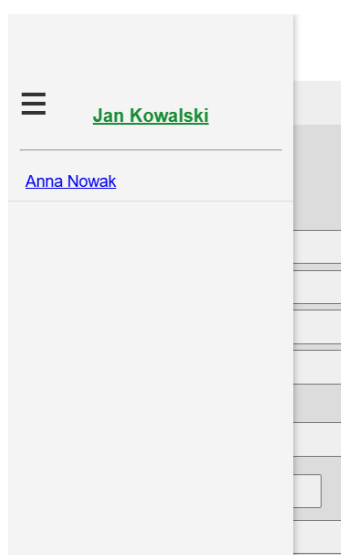
Rysunek 23: Automatyczne wyznaczenie wskaźnika CDR.

Wynik analizy zostaje natychmiast wyświetlony bezpośrednio na przycisku, umożliwiając szybki podgląd wartości bez potrzeby ręcznego zaznaczania struktur na obrazie (Rys. 23).



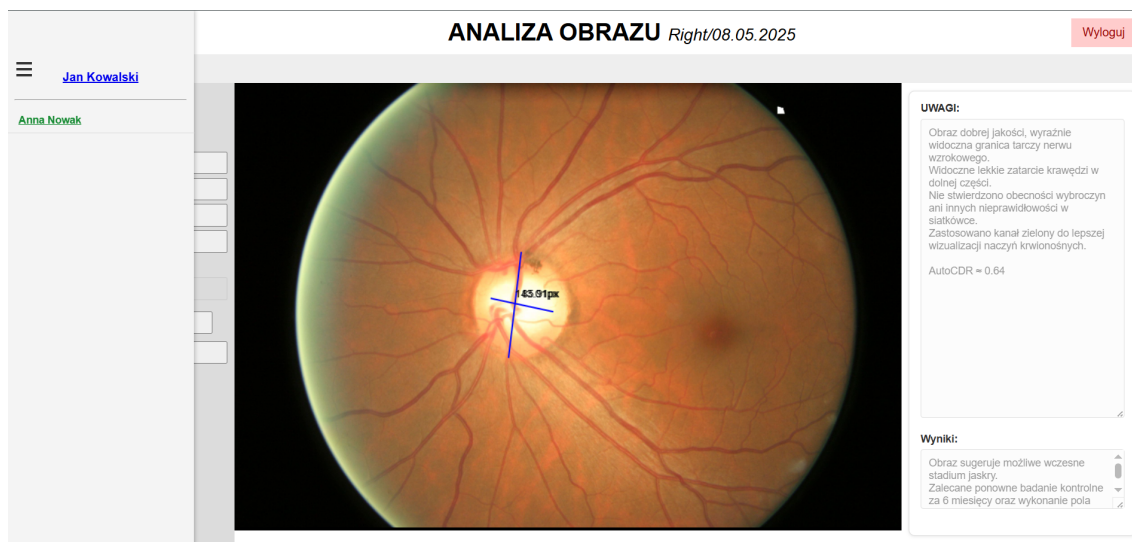
Rysunek 24: Podgląd wyników automatycznej segmentacji.

Po kliknięciu przycisku „AutoSegmentation” otwiera się okno modalne, w którym prezentowane są obrazy z automatycznie wyznaczonymi granicami tarczy nerwu wzrokowego (Optic Disc) oraz jej wgłębienia (Optic Cup). Wygenerowany wizualny wynik segmentacji umożliwia szybką ocenę jakości rozpoznanych struktur (Rys. 24).



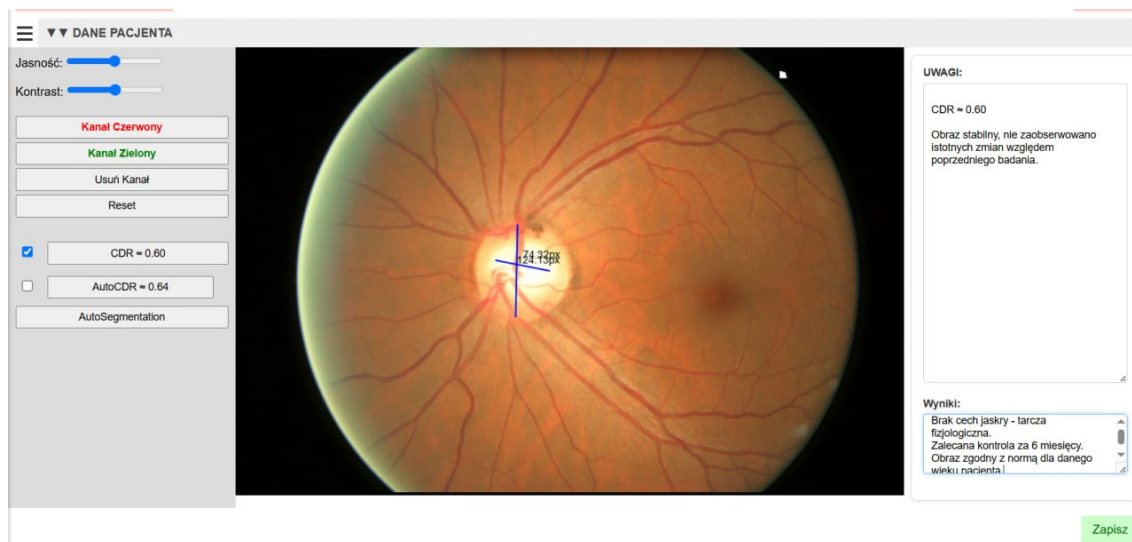
Rysunek 25: Panel lekarzy.

Panel lekarzy (sidebar) – zawiera listę lekarzy, którzy dokonali badania wybranego obrazu. Panel umożliwia przełączanie się między innymi analizami (Rys. 25).



Rysunek 26: Podgląd analizy innego lekarza.

W panelu bocznym lekarz, którego analiza jest aktualnie przeglądana, oznaczony jest kolorem zielonym. Wszystkie pola oraz przyciski umożliwiające edycję danych zostają automatycznie zablokowane, jeśli użytkownik nie jest autorem danej analizy. Dzięki temu system zapewnia integralność danych i chroni przed ich nieautoryzowaną modyfikacją (Rys. 26).



Rysunek 27: Przycisk „Zapisz”.

Przycisk „Zapisz” umieszczony w prawym dolnym rogu strony analizy obrazu umożliwia trwałe zapisanie wszystkich zgromadzonych danych dotyczących badania (Rys. 27). Po jego kliknięciu do bazy danych trafiają zarówno wartości wskaźników CDR i AutoCDR, jak i współrzędne zaznaczonych punktów granic tarczy i wgłębia na obrazie, treść wprowadzonych uwag i wyników analizy (Rys. 28). Dzięki temu przy ponownym otwarciu strony

wybranego obrazu całość wcześniej zapisanych informacji jest automatycznie wczytywana i prezentowana lekarzowi w niezmienionej formie.

	column_name text	value text
1	id	11
2	doctor_id	5
3	patient_image_id	6
4	notes	
5	description	Brak cech jaskry – tarczyca fizjologiczna.
6	cdr	0.6
7	auto_cdr	0.64
8	cdr_points	[{"x": 0.41216363509496057, "y": 0.374674474199613}, {"x": 0.40456814898384946, "y": 0.599283849199613}, {"x": 0.36984592676162725, "y": 0.4755859325329463},

Rysunek 28: Zapis analizy w bazie danych PostgreSQL.

Rysunek 29: Panel administracyjny Django.

Na Rysunku 29 pokazano panel administracyjny Django służący do dodawania nowego użytkownika. Administrator wpisuje nazwę użytkownika, hasło oraz przypisuje go do grupy „doctor”, co nadaje mu odpowiednie uprawnienia w systemie. Dzięki temu lekarze mogą logować się i analizować obrazy pacjentów, bez dostępu do ustawień administracyjnych.

6. DALSZY ROZWÓJ APLIKACJI

Zaprojektowana aplikacja stanowi funkcjonalne narzędzie wspierające lekarzy w analizie obrazów dna oka, jednak jej architektura i modułarna budowa umożliwiają dalszą rozbudowę oraz integrację z zaawansowanymi technologiami.

6.1. WYKORZYSTANIE SZTUCZNEJ INTELIGENCJI DO KLASYFIKACJI OBRAZÓW

Jednym z najważniejszych kroków rozwojowych może być rozszerzenie systemu o moduł oparty na sztucznej inteligencji, który automatycznie analizowałby obrazy dna oka w celu wstępnej klasyfikacji diagnostycznej. Zastosowanie sieci neuronowych typu CNN (Convolutional Neural Networks) umożliwiłoby identyfikację cech charakterystycznych dla jaskry na podstawie wzorców obecnych w danych obrazowych. Takie rozwiązanie mogłoby wspierać lekarza w podejmowaniu decyzji klinicznych, zapewniając jednocześnie ustandaryzowaną ocenę.

6.2. INTEGRACJA Z SYSTEMAMI SZPITALNYMI

W celu pełnej integracji z cyfrową infrastrukturą medyczną aplikacja mogłaby zostać połączona z zewnętrznymi systemami HIS (Hospital Information System) oraz PACS (Picture Archiving and Communication System). Dzięki wykorzystaniu standardów wymiany danych, takich jak HL7 FHIR czy DICOMweb, możliwe byłoby automatyczne pobieranie obrazów diagnostycznych oraz synchronizacja wyników analiz z kartoteką pacjenta.

6.3. PRZETWARZANIE ASYNCHRONICZNE

W obecnej implementacji niektóre operacje – takie jak generowanie raportu PDF lub segmentacja obrazu – są wykonywane synchronicznie, co może prowadzić do opóźnień lub blokowania żądań HTTP. Aby zwiększyć wydajność aplikacji i zapewnić płynność działania interfejsu użytkownika, przetwarzanie tych zadań mogłoby zostać przeniesione do systemu asynchronicznego kolejkowania zadań (np. z użyciem narzędzi Celery i Redis). Takie podejście umożliwi delegowanie czasochłonnych procesów do osobnych wątków roboczych, co pozwala na szybkie zwrócenie odpowiedzi do klienta i wykonanie obliczeń w tle, bez wpływu na interaktywność aplikacji.

PODSUMOWANIE

W niniejszej pracy zrealizowano założony cel, jakim było stworzenie funkcjonalnej aplikacji webowej wspierającej proces diagnostyki jaskry na podstawie obrazów dna oka. Zrealizowana aplikacja umożliwia lekarzowi wgrywanie zdjęć, wykonanie pomiarów oraz zapisywanie wyników w sposób uporządkowany i bezpieczny.

Aplikacja została poddana ręcznym testom z wykorzystaniem obrazów pozyskanych z platformy www.kaggle.com/dataset. Testy przeprowadzono w nowoczesnych przeglądarkach internetowych, potwierdzając poprawność działania zarówno funkcjonalną, jak i w zakresie poprawnego wyświetlania interfejsu użytkownika.

Zrealizowany projekt pokazuje, że technologie webowe mogą stanowić praktyczne wsparcie w procesie diagnostycznym. Aplikacja została zaprojektowana w sposób umożliwiający jej dalszą rozbudowę i integrację z innymi narzędziami medycznymi.

BIBLIOGRAFIA

- [1] YC. Tham i in. “Global prevalence of glaucoma and projections of glaucoma burden through 2040: a systematic review and meta-analysis”. W: *Ophthalmology* (2014).
- [2] <https://www.optegra.com.pl/jaskra-co-to-jest>.
- [3] Ahmed Almazroa i in. “Optic disc and optic cup segmentation methodologies for glaucoma image detection: a survey”. W: *Journal of Ophthalmology* 2015 (2015).
doi: 10.1155/2015/746439.
- [4] Andrew J Tatham i in. “The Relationship Between Cup-to-Disc Ratio and Estimated Number of Retinal Ganglion Cells”. W: *Investigative Ophthalmology & Visual Science* (2013).
- [5] https://www.w3schools.com/python/python_intro.asp.
- [6] <https://www.djangoproject.com/>.
- [7] <https://www.django-rest-framework.org/>.
- [8] <https://www.postgresql.org/about/>.
- [9] https://docs.scipy.org/doc/scipy-1.8.0/dev/contributor/quickstart_docker.html.
- [10] <https://www.geeksforgeeks.org/how-to-display-an-opencv-image-in-python-with-matplotlib/>.
- [11] <https://numpy.org/>.
- [12] Pauli Virtanen i in. “SciPy 1.0: Fundamental algorithms for scientific computing in Python”. W: *Nature Methods* 17 (2020).
- [13] <https://matplotlib.org/>.
- [14] <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [15] <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [16] <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [17] <https://getbootstrap.com/>.
- [18] <https://git-scm.com/>.
- [19] <https://code.visualstudio.com/docs>.
- [20] <https://www.freecodecamp.org/news/how-django-mvt-architecture-works/#heading-what-is-the-mvt-architecture>.
- [21] J. Nayak i in. “Automated diagnosis of glaucoma using digital fundus images”. W: *Journal of Medical Systems* (2009), s. 337–346.

Spis rysunków

1	Zdrowe oko ($CDR \approx 0.45$)	6
2	Oko z podejrzeniem jaskry ($CDR \approx 0.90$)	6
3	Diagram przypadków użycia aplikacji.	8
4	Diagram przedstawiający przepływ danych w architekturze MVT Django [20].	12
5	Struktura aplikacji.	14
6	Diagram ERD bazy danych.	16
7	Schemat blokowy przetwarzania obrazu do obliczania wskaźnika CDR [21].	17
8	Strona logowania użytkownika.	24
9	Główna strona aplikacji.	24
10	Panel z danymi pacjenta.	25
11	Panel z danymi obrazów przed sortowaniem.	26
12	Panel z danymi obrazów po sortowaniu.	26
13	Wystawienie oceny przez lekarza.	27
14	Widok tabeli obrazów w trybie wyświetlania ocen średnich.	27
15	Wygenerowany raport	28
16	Panel statystyk wartości CDR.	29
17	Strona badania obrazu.	30
18	Powiększony obraz dna oka w kanale czerwonym z naniesionymi punktami granic tarczy nerwu wzrokowego (Optic Disc).	30
19	Sekcja „Dane pacjenta”.	31
20	Panel narzędzi po lewej stronie.	31
21	Wyznaczanie wskaźnika CDR.	32
22	Podpowiedzi korzystania z narzędzi pomiarowych.	32
23	Automatyczne wyznaczenie wskaźnika CDR.	32
24	Podgląd wyników automatycznej segmentacji.	33
25	Panel lekarzy.	33
26	Podgląd analizy innego lekarza.	34
27	Przycisk „Zapisz”.	34
28	Zapis analizy w bazie danych PostgreSQL.	35

29	Panel administracyjny Django.	35
----	---------------------------------------	----

SPIS LISTINGÓW

1	Przykładowy model użyty w aplikacji.	12
2	Przykładowy widok użyty w aplikacji.	13
3	Zapisywanie oceny zdjęcia do API (<code>home.js</code>).	18
4	Widok Django zapisujący ocenę lekarza.	18
5	Widok logowania (<code>views.py</code>).	19
6	Formularz logowania (<code>login.html</code>).	20
7	Przykład zabezpieczonego widoku (<code>views.py</code>).	20
8	Warunek autoryzacji edycji danych.	20
9	Funkcja modyfikacji jasności i kontrastu obrazu.	21
10	Zastosowanie CLAHE w przetwarzaniu obrazu.	21
11	Funkcja segmentacji obrazu.	21
12	Przekształcenia morfologiczne.	23